

DingLAB USB2CAN及USB2FDCAN转换模块使用说明书v2.5

版本更新说明

版本	更新内容
v1.0	第一次更新文档
v2.0	更新固件与硬件
v2.1	更新收发示例
v2.2	加入带壳版本图片
v2.3	修改一些bug
v2.4	发布透传协议，增加USB2FDCAN内容，更新外壳图片，更改产品名称与店铺一致
v2.5	修改透传协议，增加案例

产品概述

DingLAB USB2CAN转换模块是一款高性能、小体积的USB转CAN接口设备，具有以下特点：

1. 小体积：模块尺寸仅为27mm × 35mm，便于安装。
2. 高速传输：支持 1Mbps (CAN) 的数据传输速率，满足高速CAN总线通信需求。
3. USB虚拟串口：采用USB2.0 FS通讯协议，同时使用USB Type-C接口进行数据传输和供电。
4. 两路CAN接口：具备两个独立的CAN通道，可同时连接两个CAN网络。
5. 匹配电阻：模块为CAN总线设置了120 Ω匹配电阻，并设置拨码开关方便用户选择是否接入。
6. 设备名绑定：模块具有USB设备PID设置功能。通过拨码开关设置，可在Linux上将设备设置为 USB2CAN0-USB2CAN3，消除模块插入后设备名称变化以及多个模块之间的区分问题。

DingLAB USB2FDCAN转换模块在USB2CAN基础上升级，增加了以下功能：

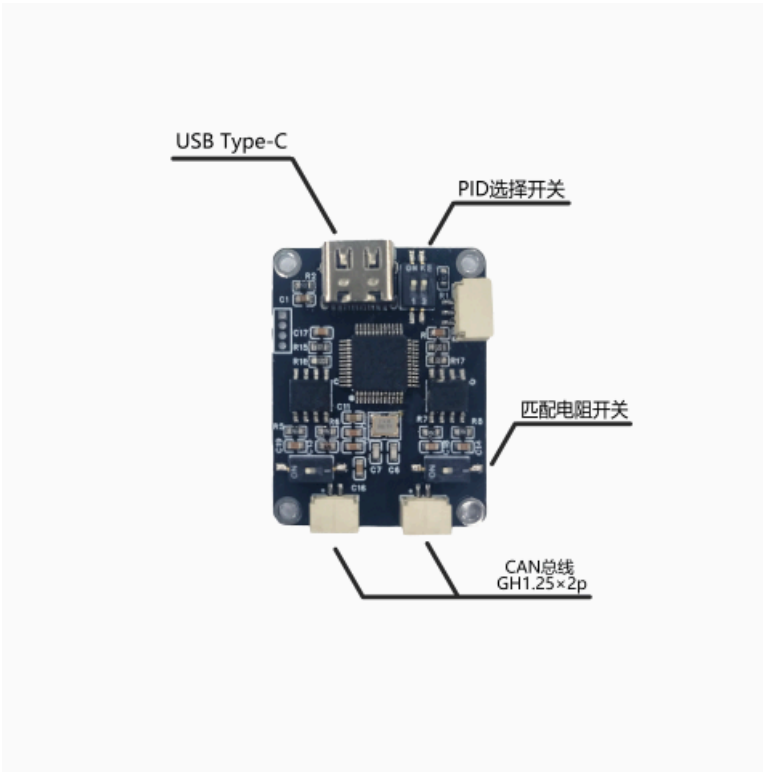
1. 支持FDCAN：兼容FDCAN协议，适应更广泛的CAN总线应用，可通过函数配置修改为 500kbps (CAN)、1Mbps (CAN)、500kbps (FDCAN)、1Mbps (FDCAN)、2Mbps (FDCAN)、4Mbps (FDCAN)。

功能介绍

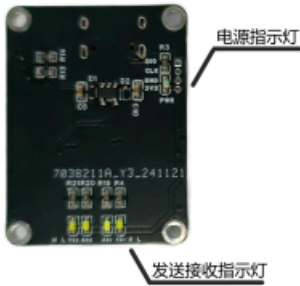
模块主要参数

使用说明

无壳版本



无壳版本，上图中，左侧为CAN1，右侧为CAN2，两路CAN线序均为左侧为CANL，右侧为CANH。



带壳版本

带壳版本，上图中，左侧为CAN2，右侧为CAN1，两路CAN线序均为左侧为CANH，右侧为CANL。
LED从左到右依次为CAN2接收、CAN2发送、CAN1发送、CAN1接收。



PID切换

用户可以使用PID切换开关切换设备PID，开关状态与自动识别的设备名称对应如下表(上表示拨到ON KE方向即朝USB口方向，下表示拨到 1 2方向)

开关1	开关2	设备名称
上	上	USB2CAN0
下	上	USB2CAN1
上	下	USB2CAN2
下	下	USB2CAN3

匹配电阻

模块为CAN总线设置了120Ω匹配电阻，并设置拨码开关方便用户选择是否接入，将CAN接口附近对应单路拨码开关设置为ON即接入匹配电阻。电阻默认接入。

LED指示灯

每一路CAN总线配有一对数据传输指示灯，数据传输时LED灯亮白光，无数据传输时LED灯熄灭。

驱动安装

目前，本模块支持在Linux系统中进行驱动安装和二次开发，未来将支持Windows。

安装至系统目录

在SDK包内，有 `install.sh` 用于系统级安装，将会把动态链接库安装至 `\usr\local\lib`，将头文件安装至 `\usr\local\include`，并自动安装 `usb_can.rules` 至 `\etc\udev\rules.d`。

同时，有 `uninstall.sh` 可以自动将安装的文件从系统中移除。

手动安装

如不希望使用脚本自动安装至系统，可以将lib文件夹中的libusb_can.so复制至工程，并手动设置动态链接，如Cmake工具链往CMakeLists.txt中添加

```
link_directories(DIR\CONTAINS\SHARELIB)
target_link_directories(${YOUR_TARGET} usb_can)
```

同时，将include文件夹中的usb_can.h复制至工程，使工程可以发现此头文件，如Cmake工具链往CMakeLists.txt中添加

```
include_directories(DIR\CONTAINS\HEADER)
```

要使用自动绑定设备名称功能，需要将usb_can.rules移动至/etc/udev/rules.d，并使udev规则生效：

```
sudo udevadm trigger
```

USB2CAN二次开发说明

接口说明

目前，模块提供C/C++接口进行模块操作，下面给出模块操作的主要结构体和接口函数

```
enum FrameType {
    STANDARD = 0,
    EXTENDED = 1
};

typedef struct FrameInfo {
    uint32_t canID; // CAN帧ID
    uint8_t frameType; // 0为标准帧，1为扩展帧，推荐使用预先定义的宏
    uint8_t dataLength; // CAN帧数据长度，范围0-8
} FrameInfo;

//!
//! \param devName 设备名称，如 '/dev/USB2CAN0'
//! \return 设备句柄，用于操作设备进行收发，-1表示打开失败
int32_t openUSBCAN(const char *devName);

//!
//! \param dev 设备句柄
//! \return 串口关闭返回值，0表示成功，-1表示失败
int32_t closeUSBCAN(int32_t dev);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 模块的CAN通道，USB2CAN-CAN-Dual可设置为1或2
//! \param info FrameInfo结构体指针，用于设置CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据
//! \return 传输成功返回数据长度，-1表示失败
```

```

int32_t sendUSBCAN(int32_t dev, uint8_t channel, FrameInfo *info, uint8_t *data);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 收到数据的CAN通道，需要提前定义变量并传入指针
//! \param info FrameInfo结构体指针，需要提前定义变量，用于接受CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据，需要提前定义变量，用于接受CAN帧数据
//! \param timeout 接收超时时间，单位为微妙。函数会阻塞等待，直到接收到数据或者超时。
//! \return 返回0表示接收成功，返回-1表示接收超时
int32_t readUSBCAN(int32_t dev, uint8_t *channel, FrameInfo *info, uint8_t *data,
int32_t timeout);

```

使用示例

下面给出一个简单的使用示例，用于演示如何使用上述接口函数进行模块操作

发送CAN帧

```

#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }

    FrameInfo info;
    info.canID = 0x123; // 设置CAN ID 为0x123
    info.frameType = STANDARD; //设置为标准帧
    info.dataLength = 8; //设置数据长度为8
    uint8_t data[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

    int32_t ret = sendUSBCAN(dev, 1, &info, data); // 从模块CAN1发送CAN帧
    if (ret != 8) {
        printf("Failed to send CAN frame\n");
        closeUSBCAN(dev);
        return -1;
    }
}

```

接受CAN帧

```

#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }
}

```

```

}

uint8_t channel; //初始化接收到的CAN通道
FrameInfo info; //初始化接收到的CAN帧信息
uint8_t data[8]; //初始化接收到的CAN帧数据

ret = readUSBCAN(dev, &channel, &info, data, 1e6); // 从模块CAN1接收CAN帧，超时
时间为1秒
if (ret == 0) {
    printf("Received CAN frame on channel %d\n", channel);
    printf("CAN ID: 0x%03X\n", info.canID);
    printf("Frame type: %s\n", info.frameType == STANDARD ? "Standard" :
"Extended");
}
closeUSBCAN(dev);
return 0;
}

```

USB2FDCAN二次开发说明

接口说明

目前，模块提供C/C++接口进行模块操作，下面给出模块操作的主要结构体和接口函数

```

enum FrameType {
    STANDARD = 0,
    EXTENDED = 1
};

enum CANType {
    CAN = 0,
    FDCAN = 1
};

enum FDCAN_Speed {
    FDCAN_500K = 0,
    FDCAN_1M,
    FDCAN_2M,
    FDCAN_4M
};

enum DLC {
    FDCAN_0BYTES = 0,
    FDCAN_1BYTES,
    FDCAN_2BYTES,
    FDCAN_3BYTES,
    FDCAN_4BYTES,
    FDCAN_5BYTES,
    FDCAN_6BYTES,
    FDCAN_7BYTES,
    FDCAN_8BYTES,
    FDCAN_12BYTES = 9,
    FDCAN_16BYTES,
    FDCAN_20BYTES,
    FDCAN_24BYTES,
    FDCAN_32BYTES,
    FDCAN_48BYTES,

```

```

FDCAN_64BYTES
};

typedef struct FrameInfo {
    uint32_t canID; // CAN帧ID
    uint8_t frameType; // 0为标准帧, 1为扩展帧, 推荐使用预先定义的宏
    uint8_t dataLength; // CAN帧数据长度, 范围0-8
} FrameInfo;

//!
//! \param devName 设备名称, 如 '/dev/USB2CAN0'
//! \return 设备句柄, 用于操作设备进行收发, -1表示打开失败
int32_t openUSBCAN(const char *devName);

//!
//! \param dev 设备句柄
//! \return 串口关闭返回值, 0表示成功, -1表示失败
int32_t closeUSBCAN(int32_t dev);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 模块的CAN通道, USB2CAN-CAN-Dual可设置为1或2
//! \param info FrameInfo结构体指针, 用于设置CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据
//! \return 传输成功返回串口帧总长度, -1表示失败
int32_t sendUSBCAN(int32_t dev, uint8_t channel, FrameInfo *info, uint8_t *data);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 收到数据的CAN通道, 需要提前定义变量并传入指针
//! \param info FrameInfo结构体指针, 需要提前定义变量, 用于接受CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据, 需要提前定义变量, 用于接受CAN帧数据
//! \param timeout 接收超时时间, 单位为微妙。函数会阻塞等待, 直到接收到数据或者超时
//! \return 返回0表示接收成功, 返回-1表示接收超时
int32_t readUSBCAN(int32_t dev, uint8_t *channel, FrameInfo *info, uint8_t *data,
int32_t timeout);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 需要配置的CAN通道
//! \param type CANType枚举体 选择配置为FDCAN或CAN
//! \param nominalSpeed FDCAN_Speed枚举体, 仲裁段速率, type选择FDCAN时可以选择500Kbps或
1Mbps; type选择CAN时要求与数据段速率选择一致
//! \param dataSpeed FDCAN_Speed枚举体, 数据段速率, type 选择FDCAN时可以选择500Kbps、
1Mbps、2Mbps、4Mbps; type选择CAN时只能选择500Kbps或1Mbps
//! \return 返回0表示配置成功, 返回-1表示配置超时
int32_t configUSBCAN(int32_t dev, uint8_t channel, uint8_t type, uint8_t
nominalSpeed, uint8_t dataSpeed);

```

使用示例

下面给出一个简单的使用示例, 用于演示如何使用上述接口函数进行模块操作

发送FDCAN帧

```
#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }
    // 配置成仲裁段500Kbps、数据段2Mbps的FDCAN
    ret = configUSBCAN(handler, 2, FDCAN, FDCAN_500K, FDCAN_2M);
    // 配置成1Mbps的CAN
    // ret = configUSBCAN(handler, 2, CAN, FDCAN_1M, FDCAN_1M);
    FrameInfo info;
    info.canID = 0x123; // 设置CAN ID 为0x123
    info.frameType = STANDARD; //设置为标准帧
    info.dataLength = FDCAN_64BYTES; //设置数据长度为64
    // info.dataLength = FDCAN_8BYTES; //设置数据长度为8
    uint8_t data[64] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

    int32_t ret = sendUSBCAN(dev, 1, &info, data); // 从模块CAN1发送CAN帧
    if (ret != 8) {
        printf("Failed to send CAN frame\n");
        closeUSBCAN(dev);
        return -1;
    }
}
```

接受FDCAN帧

```
#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }
    // 配置成仲裁段500Kbps、数据段2Mbps的FDCAN
    ret = configUSBCAN(handler, 2, FDCAN, FDCAN_500K, FDCAN_2M);
    uint8_t channel; //初始化接收到的CAN通道
    FrameInfo info; //初始化接收到的CAN帧信息
    uint8_t data[64]; //初始化接收到的CAN帧数据

    ret = readUSBCAN(dev, &channel, &info, data, 1e6); // 从模块CAN1接收CAN帧，超时
    时间为1秒
    if (ret == 0) {
        printf("Received CAN frame on channel %d\n", channel);
        printf("CAN ID: 0x%03X\n", info.canID);
    }
}
```



```
        printf("Frame type: %s\n", info.frameType == STANDARD ? "Standard" :
"Extended");
    }
    closeUSBCAN(dev);
    return 0;
}
```

串口协议

硬件配置

可以使用本模块提供的USB2CAN串口调试工具进行串口通信。使用浏览器打开html文件即可。

USB虚拟串口，无一般串口波特率等要求。

协议内容

协议格式

ID (1Byte)	数据 (nBytes)	CRC8校验 (1Byte)
决定数据功能	不定长，由ID决定	校验

CAN数据发送（上位机 -> 模块）

ID: 0xA8

数据长度: 15Bytes

例如：使用CAN通道：1，can设备ID为：0x7ff，标准帧，数据长度为：4，数据内容为：0xff 0xff 0x00 0x82

发送的数据（16进制）为：A8 01 FF 07 00 00 00 04 FF FF 00 82 E1 （最后一个字节为crc校验结果）

数据格式

通道数 (1Byte)	CAN设备ID (4Bytes)	帧类型 (1Byte)	数据长度 (1Byte)	数据 (8Bytes)
选择发送数据的CAN设备（1或2）	CAN数据帧目标设备ID，电机ID，高位在右	选择标准帧（0）或拓展帧（1）	CAN数据帧数据长度（最大为8）	要发生的数据

CAN数据接收（模块 -> 上位机）

ID: 0XA9

数据长度: 15Bytes

数据格式

通道数 (1Byte)	CAN设备ID (4Bytes)	帧类型 (1Byte)	数据长度 (1Byte)	数据 (8Bytes)
接收到数据的CAN设备（1或2）	CAN数据帧ID，电机ID，高位在右	标准帧（0）或 拓展帧（1）	收到的CAN数据帧数据长度	接收到的数据

售后服务

SOULDE Studio提供完善的售后服务，如有任何疑问或需求，请随时与我们联系。我们将竭诚为您解答和解决问题。模块出现非人为损坏质量问题，可以与店铺联系退换，来回邮费由我们承担。

感谢您选择SOULDE Studio USB2CAN转换模块，祝您使用愉快！

附录

CRC校验函数

```
/* crc.h */
#pragma once
#ifdef __cplusplus
extern "C"{
#endif
#include <stdint.h>

// CRC8
unsigned char Get_CRC8_Check_Sum(unsigned char* pchMessage, unsigned int dwLength, unsigned char uCRC8);
unsigned int Verify_CRC8_Check_Sum(unsigned char* pchMessage, unsigned int dwLength);
void Append_CRC8_Check_Sum(unsigned char* pchMessage, unsigned int dwLength);

#ifdef __cplusplus
}
#endif

/* crc.c */
#include "crc.h"
#include <stdlib.h>

#define U8 uint8_t

const unsigned char CRC8_INIT = 0xff;
const unsigned char CRC8_TAB[256] =
{
```

```

    0x00, 0x5e, 0xbc, 0xe2, 0x61, 0x3f, 0xdd, 0x83, 0xc2, 0x9c, 0x7e, 0x20, 0xa3,
    0xfd, 0x1f, 0x41,
    0x9d, 0xc3, 0x21, 0x7f, 0xfc, 0xa2, 0x40, 0x1e, 0x5f, 0x01, 0xe3, 0xbd, 0x3e,
    0x60, 0x82, 0xdc,
    0x23, 0x7d, 0x9f, 0xc1, 0x42, 0x1c, 0xfe, 0xa0, 0xe1, 0xbf, 0x5d, 0x03, 0x80,
    0xde, 0x3c, 0x62,
    0xbe, 0xe0, 0x02, 0x5c, 0xdf, 0x81, 0x63, 0x3d, 0x7c, 0x22, 0xc0, 0x9e, 0x1d,
    0x43, 0xa1, 0xff,
    0x46, 0x18, 0xfa, 0xa4, 0x27, 0x79, 0x9b, 0xc5, 0x84, 0xda, 0x38, 0x66, 0xe5,
    0xbb, 0x59, 0x07,
    0xdb, 0x85, 0x67, 0x39, 0xba, 0xe4, 0x06, 0x58, 0x19, 0x47, 0xa5, 0xfb, 0x78,
    0x26, 0xc4, 0x9a,
    0x65, 0x3b, 0xd9, 0x87, 0x04, 0x5a, 0xb8, 0xe6, 0xa7, 0xf9, 0x1b, 0x45, 0xc6,
    0x98, 0x7a, 0x24,
    0xf8, 0xa6, 0x44, 0x1a, 0x99, 0xc7, 0x25, 0x7b, 0x3a, 0x64, 0x86, 0xd8, 0x5b,
    0x05, 0xe7, 0xb9,
    0x8c, 0xd2, 0x30, 0x6e, 0xed, 0xb3, 0x51, 0x0f, 0x4e, 0x10, 0xf2, 0xac, 0x2f,
    0x71, 0x93, 0xcd,
    0x11, 0x4f, 0xad, 0xf3, 0x70, 0x2e, 0xcc, 0x92, 0xd3, 0x8d, 0x6f, 0x31, 0xb2,
    0xec, 0x0e, 0x50,
    0xaf, 0xf1, 0x13, 0x4d, 0xce, 0x90, 0x72, 0x2c, 0x6d, 0x33, 0xd1, 0x8f, 0x0c,
    0x52, 0xb0, 0xee,
    0x32, 0x6c, 0x8e, 0xd0, 0x53, 0x0d, 0xef, 0xb1, 0xf0, 0xae, 0x4c, 0x12, 0x91,
    0xcf, 0x2d, 0x73,
    0xca, 0x94, 0x76, 0x28, 0xab, 0xf5, 0x17, 0x49, 0x08, 0x56, 0xb4, 0xea, 0x69,
    0x37, 0xd5, 0x8b,
    0x57, 0x09, 0xeb, 0xb5, 0x36, 0x68, 0x8a, 0xd4, 0x95, 0xcb, 0x29, 0x77, 0xf4,
    0xaa, 0x48, 0x16,
    0xe9, 0xb7, 0x55, 0x0b, 0x88, 0xd6, 0x34, 0x6a, 0x2b, 0x75, 0x97, 0xc9, 0x4a,
    0x14, 0xf6, 0xa8,
    0x74, 0x2a, 0xc8, 0x96, 0x15, 0x4b, 0xa9, 0xf7, 0xb6, 0xe8, 0x0a, 0x54, 0xd7,
    0x89, 0x6b, 0x35
};

```

```

unsigned char Get_CRC8_Check_Sum(unsigned char *pchMessage,
                                unsigned int dwLength, unsigned char ucCRC8) {
    unsigned char ucIndex;
    while (dwLength--) {
        ucIndex = ucCRC8 ^ (*pchMessage++);
        ucCRC8 = CRC8_TAB[ucIndex];
    }
    return (ucCRC8);
}

```

```

/*
** Descriptions: CRC8 Verify function
** Input: Data to verify, Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
*/
unsigned int Verify_CRC8_Check_Sum(unsigned char *pchMessage, unsigned int
dwLength) {
    unsigned char ucExpected = 0;
    if ((pchMessage == 0) || (dwLength <= 2)) return 0;
    ucExpected = Get_CRC8_Check_Sum(pchMessage, dwLength - 1, CRC8_INIT);
    return (ucExpected == pchMessage[dwLength - 1]);
}

```

```
/*
** Descriptions: append CRC8 to the end of data
** Input: Data to CRC and append,Stream length = Data + checksum
** Output: True or False (CRC Verify Result)
*/
void Append_CRC8_Check_Sum(unsigned char *pchMessage, unsigned int dwLength) {
    unsigned char ucCRC = 0;
    if ((pchMessage == 0) || (dwLength <= 2)) return;
    ucCRC = Get_CRC8_Check_Sum((unsigned char *) pchMessage, dwLength - 1,
CRC8_INIT);
    pchMessage[dwLength - 1] = ucCRC;
}
```